



北京邮电大学

Beijing University of Posts and Telecommunications

移动终端安全

课程报告

标 题

第一次作业

ARM 安全技术调研与 ATF 虚拟环境搭建

姓 名

学 号

班 级

教 师

移动终端安全第一次作业

目 录

一、ARM 安全技术调研	3
1.1 目标.....	3
1.2 核心服务原理分析.....	4
1.3 源代码分析.....	6
1.3.1 BL1 与 BL2 中的认证初始化	6
1.3.2 load_auth_image() 与递归认证流程	8
1.3.3 CoT 描述文件与信任链定义	9
1.3.4 auth_mod_verify_img() 的认证调度机制	12
1.3.5 auth_signature() 的运行机制	14
1.3.6 auth_nvctr() 的防回滚机制.....	16
1.3.7 mbedTLS 中的底层验签与哈希计算.....	16
1.3.8 BL1 认证 BL2 的完整运行过程总结	17
1.3.9 流程图.....	18
1.4 分析结论.....	21
二、思考题	22
2.1 目标.....	22
2.2 相关实验过程.....	22
2.2.1 安装前期依赖.....	22
2.2.2 查看搭建的 ATF 虚拟运行环境	29
2.2.3 登录 Secure World 的 linux 系统并启动命令行 shell.....	30
2.2.4 登录 Normal World 的 linux 系统并启动命令行 shell.....	33
2.3 思考题总结.....	35

一、ARM 安全技术调研

1.1 目标

本报告分析 ARM Trusted Firmware-A，也就是 ATF 中的可信启动与签名认证机制。选择这一核心服务，主要是因为在整个 ATF 体系里，可信启动处于非常基础但又非常关键的位置。课程中讲到，ARM TrustZone 的核心思想是把系统划分为安全世界和普通世界，让安全敏感的代码优先在安全世界中建立可信基础。而在实际工程实现中，ATF 恰恰承担了这个“建立可信基础”的职责。也就是说，TrustZone 提供了安全隔离环境，而 ATF 则在这个隔离环境中负责启动早期的安全控制、镜像认证和信任传递。如果系统在最早的启动阶段就已经加载了被篡改的镜像，那么后续无论是安全世界还是普通世界中的各种防护机制都会失去意义。因此，可信启动实际上是整个系统安全链条的起点，也是理解 ATF 安全设计最合适的切入点。

本次报告重点分析的具体对象，是 BL1 阶段对 BL2 镜像的认证加载过程，以及 BL2 阶段对后续镜像继续执行认证时所依赖的通用认证框架。通过这一主线，可以把 ATF 中“启动入口”“镜像加载”“认证框架”“信任链描述”“签名验证”和“防回滚保护”几个关键部分串联起来，从源代码层面说明 ATF 是如何一步一步建立启动信任链的。

本报告对应的关键源代码位置如下。BL1 和 BL2 的启动入口分别位于 bl1/bl1_main.c 和 bl2/bl2_main.c。这两个文件负责启动阶段的初始化工作，并且明确调用密码模块和认证模块。ATF 通用的镜像认证加载入口位于 common/bl_common.c，其中 load_auth_image()和 load_auth_image_recursive() 体现了“先认证父节点，再认证当前节点”的递归信任链思路。认证框架本身的核心代码位于 drivers/auth/auth_mod.c，其中 auth_mod_verify_img() 是总调度函数，auth_signature() 负责签名认证，auth_nvctr() 负责 NV Counter 校验。底层密码学实现位于 drivers/auth/mbedtls/mbedtls_crypto.c，这里通过 mbedtls 完成真正的哈希计算与签名验证。除此之外，ATF 中用于定义信任链关系的 CoT 文件位于 drivers/auth/tbbr/tbbr_cot_common.c、drivers/auth/tbbr/tbbr_cot_bl1.c 和 drivers/auth/tbbr/tbbr_cot_bl2.c。这些文件定义了每个证书和镜像的父子关系、认证方式以及认证成功后要提取的安全数据，是整个可信启动机制的策略基

础。

从分析目标上看，本报告希望解决以下几个问题：第一，ATF 为什么要把可信启动设计成证书链和镜像哈希相结合的方式；第二，ATF 在源码中是如何组织镜像递归认证流程的；第三，平台根信任公钥、证书、公钥、哈希和 NV Counter 是如何在代码中共同参与认证的；第四，这种设计对 TrustZone 安全启动有什么意义。通过回答这些问题，可以更清楚地理解 ATF 并不是单纯的“启动固件”，而是一套在安全世界中为系统建立可信根的关键安全软件。

1.2 核心服务原理分析

ATF 中的可信启动机制，本质上是一种围绕“根信任”展开的链式认证机制。所谓链式认证，指的是系统在启动时不会直接无条件地相信某个固件镜像，而是先验证它的上一级证书或父对象，确认上一级对象可信之后，再由上一级对象中携带的公钥、哈希或其他认证参数去验证下一级对象。这样一层一层向下扩展，最终就形成一条从平台根信任一直延伸到后续所有关键启动镜像的信任链。在理论上，这种设计的目的就是把“平台预置信任”安全地传递给后续将要运行的固件，从而保证整个启动过程不会被非授权代码劫持。

如果从安全逻辑上概括，ATF 的可信启动机制实际上解决了三个问题。第一个问题是“镜像来源是否可信”，这主要依靠数字签名来保证。签名认证的本质是：只有持有合法私钥的一方，才能对证书或镜像元数据进行签名，因此如果验签通过，就能证明这个对象的发布者是受信任的。第二个问题是“镜像内容是否完整”，这主要依靠哈希来保证。证书中会保存后续镜像的期望哈希值，而系统在加载镜像后会对实际内容重新计算哈希并进行比较，如果结果一致，就说明镜像在存储和加载过程中没有被篡改。第三个问题是“镜像是否属于允许加载的最新或合理版本”，这主要依靠 NV Counter 解决。即使某个旧版本固件的签名仍然合法，如果它已经落后于平台记录的安全版本，那么依然应当被拒绝，以防止回滚攻击。

ATF 之所以采用“证书签名认证 + 镜像哈希校验 + NV Counter 防回滚”这种组合方式，而不是只做单一的签名验证，是有很强工程合理性的。首先，对每一个原始镜像都直接附带签名并逐个处理，会增加镜像处理复杂度，也不利于把不同镜像的公钥和认证策略统一组织起来。证书机制的引入，使 ATF 可以把

多个后续镜像的哈希、公钥和相关认证参数集中封装在证书中统一保护。只要证书本身通过签名验证,那么证书中提取出的这些数据也就被视为可信。这样一来,后续原始镜像就不必各自再做复杂签名解析,而只需要对照证书中的可信哈希进行校验即可。其次,NV Counter 作为版本控制机制,又进一步弥补了单纯签名认证无法防御回滚攻击的问题。正因如此,这种三者结合的结构在启动安全中非常常见,也体现出 ATF 在安全与工程复杂度之间取得了较好的平衡。

从 ATF 内部结构看,可信启动并不是一个孤立模块,而是和多个子模块共同组成一套完整的启动安全服务。首先,它和 BL1、BL2 的启动流程高度耦合。BL1 作为系统上电后最早进入的可信固件阶段,在完成平台初始化后,会立即初始化密码模块 `crypto_mod_init()` 和认证模块 `auth_mod_init()`,然后在加载 BL2 时走认证加载接口。BL2 在进入后也是类似的,它会再次初始化认证框架,然后继续认证 BL31、BL32、BL33 等后续镜像。这说明 ATF 的认证机制不是附加在启动流程之后的检查步骤,而是内嵌在启动流程中的基础控制逻辑。换句话说,启动和认证在 ATF 中实际上是同步推进的。

其次,可信启动与镜像加载模块的关系非常紧密。`common/bl_common.c` 中的 `load_auth_image()` 将镜像读取、镜像认证和错误处理放在一起,这表明在启用可信启动的构建配置下,ATF 不允许“先加载、后决定是否认证”的宽松模式,而是要求加载与认证强绑定。更关键的是,ATF 使用 `load_auth_image_recursive()` 来递归认证父节点,这使得信任链可以通过通用方式被建立,而不需要为每个镜像单独手写一套固定认证路径。这样的设计既符合理论上的链式信任模型,也让代码在工程上保持较高的复用性。

再次,CoT 描述模块在整个系统中承担的是“认证策略定义层”的角色。ATF 并没有把“BL2 必须做哈希验证”“某个证书必须做签名和计数器校验”这些规则硬编码在认证框架里,而是把这些策略写进 CoT 描述文件中。每个对象都通过 `auth_img_desc_t` 结构定义自己的类型、父节点、认证方法和认证成功后要输出的可信数据。认证框架运行时只需要读取这些描述,就可以知道当前对象应当如何处理。这种做法的好处非常明显:一方面,同一套认证框架可以服务于不同平台和不同的镜像布局;另一方面,系统的信任关系和安全策略被清晰地放到了单独文件中,更有利于理解和维护。

此外，可信启动模块还与底层密码学库和平台接口密切协同。ATF 并不重复实现底层密码算法，而是使用 mbedTLS 完成 ASN.1 解析、公钥解析、摘要计算和数字签名验证。认证框架的职责不是重新实现 RSA 或 ECDSA，而是构造正确的认证上下文，提取需要参与验签的数据，然后交给密码库完成真正的数学运算。与此同时，ATF 还必须依赖平台提供的安全接口。对于根证书来说，系统必须通过 `plat_get_rotpk_info()` 获取平台中预置的根信任公钥或其哈希，才能建立真正的硬件级信任根。对于版本控制来说，系统又要通过 `plat_get_nv_ctr()` 获取平台当前记录的 NV Counter，才能判断证书是否属于可接受的版本。可见，ATF 的可信启动虽然在软件层实现，但其根基仍然来自平台硬件和安全存储能力。

如果结合课程中讲述的 TrustZone 架构来看，ATF 的可信启动机制还具有一层更重要的意义。TrustZone 提供的并不只是“安全世界可以运行安全代码”这么简单，更重要的是，安全世界本身也必须先被正确建立。如果安全世界启动时就已经载入了被攻击者篡改的 BL2、BL31 或其他安全组件，那么 TrustZone 的隔离能力本身就可能被错误使用。因此，可信启动的真正价值，在于它先保证安全世界启动链本身可信，然后才让安全世界去承载后续的各种安全服务。可以说，ATF 的可信启动是 TrustZone 安全世界得以成立的前提条件之一。

总体来看，ATF 中的可信启动并不是一个简单的验签功能，而是贯穿 BL1、BL2 启动过程的安全控制体系。它通过根信任、公钥认证、镜像哈希、递归父子关系和防回滚机制，共同建立了一个较为严密的启动信任模型。也正因为如此，可信启动在 ATF 中并不只是“一个功能模块”，而是整个可信固件架构的安全核心。

1.3 源代码分析

本部分围绕 BL1 认证 BL2 的主线，对 ATF 中最关键的代码进行分析。分析顺序按照启动链的自然执行顺序展开，也就是先看 BL1/BL2 启动入口，再看通用镜像认证入口，随后分析 CoT 结构、认证调度函数、签名认证函数、NV Counter 检查函数，以及底层 mbedTLS 的实现。这样能够让源码分析尽量连贯，而不是碎片化地只解释某几个单独函数。

1.3.1 BL1 与 BL2 中的认证初始化

在 bl1/bl1_main.c 中，BL1 的主流程在完成平台和体系结构初始化之后，会执行如下代码：

```
crypto_mod_init();  
/* Initialize authentication module */  
auth_mod_init();
```

这段代码的作用并不只是完成普通初始化，而是明确告诉我们：BL1 在开始加载 BL2 之前，必须先准备好两个基础能力。第一个是密码能力，也就是后续摘要计算、签名验证和公钥相关处理所依赖的底层密码模块；第二个是认证能力，也就是镜像解析、CoT 读取、认证参数提取和认证状态维护所依赖的认证框架。换句话说，从 BL1 的设计上就已经可以看出，ATF 并不认为加载镜像是一个可以先做完、之后再检查的过程，而是要求系统在启动最早阶段就具备对镜像进行安全判断的能力。

随后 BL1 调用：

```
bl1_load_bl2();
```

在 bl1_load_bl2() 中，最关键的一句是：

```
err = load_auth_image(BL2_IMAGE_ID, info);
```

这句代码清楚地说明，BL1 并不是简单地“去存储设备读取 BL2”，而是通过“认证加载”接口来处理 BL2。这个接口名本身就体现出一种安全设计理念：镜像加载本身就带有认证语义。也就是说，只有加载并认证共同成功，BL2 才能被视为可继续执行的镜像。这种设计非常符合安全启动的原则，也避免了普通系统中可能出现的“先把镜像拿进来再说”的宽松做法。

在 bl2/bl2_main.c 中也可以看到相似的逻辑。BL2 在自己的启动阶段同样会调用 crypto_mod_init() 和 auth_mod_init()，然后进入 bl2_load_images()，继续认证后续 BL31、BL32、BL33 等镜像。这说明 ATF 中的可信启动不是只发生在 BL1 到 BL2 之间的一次性动作，而是会随着启动链向后不断延伸。BL1 对 BL2 的认证只是整条信任链的起点，BL2 对后续镜像的认证则是这条信任链的继续传播。

从安全意义上说，这段初始化代码说明了一件很重要的事：ATF 把认证模块看成启动流程的内建组成部分，而不是附属插件。这一点在写课程报告时非常值

得强调，因为它体现出 ATF 和普通固件加载逻辑的差异。普通启动加载更多关注“能不能把镜像读出来并跳转”，而 ATF 更关注“镜像在被读出来之后是否值得相信”。

1.3.2 load_auth_image() 与递归认证流程

在 common/bl_common.c 中，load_auth_image() 是 ATF 在可信启动场景下进行镜像加载的总入口，而真正体现链式认证思想的核心代码位于 load_auth_image_recursive() 中。下面这段代码最值得关注：

```
rc = auth_mod_get_parent_id(image_id, &parent_id);
if (rc == 0) {
    rc = load_auth_image_recursive(parent_id, image_data);
    if (rc != 0) {
        return rc;
    }
}
```

这段代码意味着，ATF 在处理当前镜像之前，并不会立即去加载和校验当前镜像，而是先问一个问题：这个镜像有没有父节点。如果有，就必须先去认证父节点。这个逻辑看起来很简单，但它实际上就是“

链式信任”在代码层面的核心体现。比如，BL2 在 BL1 阶段并不是直接由平台根公钥认证的，它的父节点是 trusted_boot_fw_cert。因此，ATF 在准备认证 BL2 时，会先递归进入根证书的认证流程，确保根证书可信，之后再回到 BL2 本身的认证过程。

这一递归设计有两个优点。第一，它让代码结构非常自然。对于任意一个镜像或证书来说，系统都不需要手工写死“它之前必须先验证谁”，而是只需要通过 CoT 查询父节点，然后递归向上即可。第二，它保证了信任的建立顺序一定是自上而下的。也就是说，只有当一个对象的父节点已经被确认可信之后，这个对象才有资格被认证。这样就避免了在逻辑上出现“先使用了一个尚未建立可信性的父对象”这种安全漏洞。

在完成父节点递归认证之后，代码继续执行当前对象的加载和认证：

```
rc = load_image(image_id, image_data);
```



```
if (rc != 0) {  
    return rc;  
}  
rc = auth_mod_verify_img(image_id,  
                          (void *)image_data->image_base,  
                          image_data->image_size);
```

从这里可以看出，ATF 把处理过程分成两步。第一步是把镜像或者证书加载到内存中；第二步是立刻调用认证框架对其内容进行验证。这里之所以必须“先加载再认证”，是因为哈希比对和签名认证都需要基于实际镜像内容进行计算。但与此同时，ATF 又没有把这两步分得很开，而是紧紧串联在一起，从而保证加载后的对象不会在未认证状态下进入后续流程。

更关键的是，如果认证失败，ATF 并不是简单返回错误，而是执行如下处理：

```
zero_normalmem((void *)image_data->image_base,  
               image_data->image_size);  
flush_dcache_range(image_data->image_base,  
                   image_data->image_size);  
return -EAUTH;
```

这部分逻辑非常值得重点分析。认证失败说明当前对象不可信，而不可信对象最大的风险之一，是它仍然残留在内存中，可能在异常处理、错误恢复、调试接口或者某些意外路径中被再次引用。ATF 选择在认证失败后立即将镜像内容清零，并刷新缓存，实质上是在最大程度地降低未认证对象被继续利用的机会。这个处理细节虽然代码量不大，但它体现出可信启动不是单纯的“判断通过还是失败”，而是连失败后的状态清理都考虑到了，这是一种更完整的安全工程思路。

因此，从 `load_auth_image_recursive()` 可以看出，ATF 的可信启动在镜像加载层面至少体现了三条重要原则：第一，认证顺序必须沿父节点递归建立；第二，镜像加载必须与镜像认证紧密绑定；第三，认证失败的对象必须立即从内存中清除。单从这一部分源码，就已经能看出 ATF 在启动安全上的设计相当成熟。

1.3.3 CoT 描述文件与信任链定义

ATF 中用于定义信任链关系的核心代码位于

drivers/auth/tbbr/tbbr_cot_common.c、drivers/auth/tbbr/tbbr_cot_bl1.c 和 drivers/auth/tbbr/tbbr_cot_bl2.c。这些文件的重要性在于，它们并不是简单地列出镜像 ID，而是在源码层定义了“谁信任谁”“用什么信任”“信任成功后留下什么数据”。

在 tbbr_cot_common.c 中，可以看到根证书 trusted_boot_fw_cert 的定义：

```
const auth_img_desc_t trusted_boot_fw_cert = {
    .img_id = TRUSTED_BOOT_FW_CERT_ID,
    .img_type = IMG_CERT,
    .parent = NULL,
    .img_auth_methods = (const
auth_method_desc_t[AUTH_METHOD_NUM]) {
    [0] = {
        .type = AUTH_METHOD_SIG,
    },
    [1] = {
        .type = AUTH_METHOD_NV_CTR,
    }
},
    .authenticated_data = ...
};
```

这个结构中最关键的几个字段值得分别解释。首先，`.parent=NULL` 表示它没有父节点，因此它是当前信任链的根对象。在 ATF 的认证模型里，没有父节点的对象并不意味着“天然可信”，而意味着它必须直接依赖平台中的根信任来建立可信性。也就是说，对于 `trusted_boot_fw_cert` 这类根证书，系统不可能从上一级证书中提取公钥，因为它本身已经是最上层证书了，所以只能借助平台中预置的 ROTPK 或其哈希作为最终信任源。其次，`AUTH_METHOD_SIG` 表示该证书必须执行签名认证。这是为了确认这个证书确实来自受信任的签发方，且证书内容没有被篡改。再次，`AUTH_METHOD_NV_CTR` 表示这个证书还必须进行版本计数器检查，从而防止攻击者回滚到旧版本的合法证书。最后，

`authenticated_data` 表明这个证书认证成功后并不是只返回一个布尔结果，而是还要把其中保存的多个哈希值或其他认证参数提取出来，为后续下级镜像认证服务。这一点很重要，因为它揭示了证书在 ATF 中的真正角色：证书不仅是被认证对象，本身也是下级对象认证所依赖的可信数据容器。

在 `tbbr_cot_bl1.c` 中，BL2 的认证描述如下：

```
static const auth_img_desc_t bl2_image = {
    .img_id = BL2_IMAGE_ID,
    .img_type = IMG_RAW,
    .parent = &trusted_boot_fw_cert,
    .img_auth_methods = (const
auth_method_desc_t[AUTH_METHOD_NUM]) {
        [0] = {
            .type = AUTH_METHOD_HASH,
        }
    }
};
```

这里首先说明，BL2 的父节点是 `trusted_boot_fw_cert`，因此 BL2 的可信性并不是直接来源于平台硬件根信任，而是来源于已经认证通过的根证书。其次，BL2 的认证方式是 `'AUTH_METHOD_HASH'`，不是 `'AUTH_METHOD_SIG'`。这意味着 BL2 本身作为原始镜像，不直接参与复杂签名结构的解析和验签流程，而是只对照根证书中提取出来的可信哈希进行内容完整性校验。这种“证书负责建立信任，镜像负责接受哈希校验”的模式，是 ATF 中非常典型的做法。

继续查看 `tbbr_cot_bl2.c`，可以发现 BL31、BL32、BL33 及其对应证书，也都是用类似方式组织起来的。例如 `trusted_key_cert`、`soc_fw_key_cert`、`soc_fw_content_cert`、`non_trusted_fw_key_cert`、`non_trusted_fw_content_cert` 等对象之间都通过 `'parent'` 连接成更长的信任链。也就是说，BL1 到 BL2 只是一个最简单的起点，到了 BL2 阶段，这条信任链会继续向 BL31、BL32、BL33 扩展，形成更完整的启动认证树。

从设计角度看，CoT 文件的重要意义在于，它把安全策略和认证框架分离开

了。认证框架本身不需要预先知道“BL2 的父节点是什么”“BL33 是哈希认证还是签名认证”，它只需要读取 CoT 中的定义，就能决定如何处理当前对象。这种设计极大提高了可扩展性，也让整个认证关系在源码层面更加清晰。对于课程报告来说，这也是 ATF 很值得分析的一点，因为它不仅体现了安全设计，还体现了良好的工程组织方式。

1.3.4 auth_mod_verify_img() 的认证调度机制

在 drivers/auth/auth_mod.c 中，auth_mod_verify_img() 是 ATF 认证框架中最核心的控制函数。它的任务不是亲自完成所有密码运算，而是根据 CoT 描述组织整个认证流程。函数开始时最关键的一步是：

```
img_desc = FCONF_GET_PROPERTY(tbbr, cot, img_id);
```

这句代码的意义在于，它根据当前镜像 ID 取出这个对象在 CoT 中对应的描述。也就是说，认证框架在执行前，首先要知道当前对象的身份、类型、父子关系和认证策略。只有这样，后续认证逻辑才有明确依据。这一步体现出 ATF 认证框架是“数据驱动”而不是“流程硬编码”的：真正决定认证方式的不是函数里写死的 if-else，而是 CoT 描述中给出的策略。

随后函数调用：

```
rc = img_parser_check_integrity(img_desc->img_type, img_ptr,
img_len);
```

这里是认证前的基本完整性检查。换句话说，在进行签名和哈希校验之前，ATF 先确认输入对象至少是“可以被正确解析的”。如果一个证书连 DER 结构都已经损坏，或者一个镜像结构本身已经不完整，那么继续进入后续认证逻辑就没有意义，甚至可能引发错误解析。因此，这一步虽然不是主要的安全验证，却是整个流程顺利且安全执行的必要前提。

再往下，函数会遍历当前对象在 CoT 中声明的认证方法：

```
switch (auth_method->type) {
case AUTH_METHOD_HASH:
    ...
case AUTH_METHOD_SIG:
    ...
```

```
case AUTH_METHOD_NV_CTR:
    ...
}
```

这部分逻辑是整个认证框架的中枢。它说明 ATF 并不把对象分成某种固定类别后套用固定流程，而是根据配置逐一执行声明过的认证方法。这样一来，同一个对象可以同时具备多种认证约束。例如根证书需要先通过签名认证，再通过 NV Counter 检查；而 BL2 这种原始镜像则只需要进行哈希验证。通过这种结构，ATF 把“认证对象”和“认证方式”彻底解耦开来，使同一套框架可以适配多种不同对象。

在所有声明的认证方法都成功后，函数并不会立即结束，而是继续从当前对象中提取后续认证需要使用的数据：

```
rc = img_parser_get_auth_param(...);
memcpy((void *)img_desc->authenticated_data[i].data.ptr,
        (void *)param_ptr, param_len);
```

这一部分代码非常关键，却很容易被忽略。它表明认证的目标并不只是得出“通过”或“失败”的结论，而是为了把当前对象中的可信数据向下一级传递。例如根证书认证成功后，ATF 会从中提取 BL2 的哈希值、配置镜像的哈希值，或者下一级证书所需的公钥。认证框架把这些值复制到指定缓冲区后，下一级对象在认证时就可以直接使用这些已经建立可信性的参数。也就是说，`auth_mod_verify_img()` 不只是判断当前对象是否可信，更是在“认证成功的同时生产下一级认证所需的可信输入”。

最后，函数用如下语句记录当前对象已经被认证：

```
auth_img_flags[img_desc->img_id] |= IMG_FLAG_AUTHENTICATED;
```

这个状态标记意味着，当前对象以后可以作为父对象参与子对象认证，而且框架能够知道它已经是可信状态，避免重复或错误处理。综合来看，`auth_mod_verify_img()` 的作用可以概括为四点：根据 CoT 找到认证策略、对对象执行相应认证方法、提取可信数据供子对象使用、记录当前对象的认证状态。它并不直接实现底层密码学，但却承担了整个认证逻辑的组织与控制，因此是 ATF 可信启动代码中最核心的函数之一。

1.3.5 auth_signature() 的运行机制

在 auth_mod.c 中，负责签名认证的关键函数是 auth_signature()。这个函数最先做的事情，是从当前对象中提取出签名认证所需的参数：

```
rc      =      img_parser_get_auth_param(img_desc->img_type,
param->data,
        img, img_len, &data_ptr, &data_len);
rc      =      img_parser_get_auth_param(img_desc->img_type,
param->sig,
        img, img_len, &sig_ptr, &sig_len);
rc      =      img_parser_get_auth_param(img_desc->img_type,
param->alg,
        img, img_len, &sig_alg_ptr, &sig_alg_len);
```

这部分逻辑体现出 ATF 的认证框架并不直接假设输入一定是某种固定格式的裸签名结构，而是先借助镜像解析模块把需要参与认证的各类参数提取出来。这里主要包括三类信息：一是被签名的数据本体，二是签名值，三是签名算法的描述信息。通过这种方式，认证框架和底层数据格式解耦了，框架只关心“我拿到了哪些认证参数”，而不需要自己去硬编码所有证书格式细节。

提取参数之后，`auth\_signature()` 最重要的工作是确定“应该用谁的公钥来验签”。核心代码如下：

```
if (img_desc->parent != NULL) {
    rc = auth_get_param(param->pk, img_desc->parent, &pk_ptr,
&pk_len);
} else {
    rc = plat_get_rotpk_info(param->pk->cookie, &pk_plat_ptr,
        &pk_plat_len, &flags);
}
```

这段代码完整体现了链式信任的逻辑。如果当前对象有父节点，那么用于验签的公钥不应该凭空出现，而应该从父对象中提取。因为父对象既然已经被认证通过，那么它内部携带的公钥也就具有了可信性；如果当前对象没有父节点，那

么它就是信任链的根对象，此时只能依赖平台中预置的根信任公钥 ROTPK 来建立可信性。这个判断在理论上非常自然，但在工程上实现得很优雅，因为它把根对象和普通对象统一到了一个函数中处理。

对于根证书，还要额外完成一步非常关键的工作，那就是验证证书中的公钥是否与平台预置的根信任相一致。相关代码逻辑如下：

```
if ((flags & ROTPK_IS_HASH) != 0U) {
    rc = crypto_mod_convert_pk(pk_ptr, pk_len, &cnv_pk_ptr,
    &cnv_pk_len);
    rc = crypto_mod_verify_hash(cnv_pk_ptr, cnv_pk_len,
                                pk_plat_ptr, pk_plat_len);
} else {
    if ((pk_len != pk_plat_len) ||
        (memcmp(pk_plat_ptr, pk_ptr, pk_len) != 0)) {
        return -1;
    }
}
```

这里体现出平台实现的差异性。某些平台会直接保存完整公钥，某些平台只保存公钥哈希。ATF 不强行要求平台必须使用某一种存储方式，而是通过标志位判断之后采用相应比较方式。如果平台保存的是哈希，就先对证书中的公钥进行转换和哈希，再比较；如果平台保存的是完整公钥，就直接比较公钥内容。这种设计既兼顾了不同平台的实现需求，也保证了根证书中的公钥必须和平台硬件信任根一致。

这一过程的安全意义非常大，因为它回答了一个核心问题：为什么根证书的签名不能简单地拿证书里自己的公钥去验证。如果没有这一步，攻击者完全可以伪造一个自带恶意公钥的自签名证书，只要系统盲目相信证书中包含的公钥，就会被绕过。而 ATF 的做法是，根证书中的公钥必须先和平台根信任进行绑定，绑定成功后才允许继续验签。这样才能真正保证整条信任链起点可信。

在完成上述准备之后，auth_signature() 最终调用：

```
rc = crypto_mod_verify_signature(data_ptr, data_len,
```

```
sig_ptr, sig_len,  
sig_alg_ptr, sig_alg_len,  
pk_ptr, pk_len);
```

这说明 `auth_signature()` 真正承担的是“准备验签环境”的角色。它负责解析参数、决定公钥来源、建立根信任上下文，而底层真正的密码学验签由后续模块完成。也就是说，这个函数在整个系统中的地位类似于一个安全控制中枢，它连接了 CoT 描述、平台根信任和底层密码库，是 ATF 可信启动逻辑里非常关键的桥梁。

1.3.6 `auth_nvctr()` 的防回滚机制

在 `auth_mod.c` 中，`auth_nvctr()` 用于完成 NV Counter 校验。其思路并不复杂，但安全意义很强。函数会先从当前证书中解析出计数器值，这个值通常以 DER 编码形式存在；随后它会调用平台接口读取平台当前记录的 NV Counter；最后比较两者大小。如果证书中的计数器值小于平台当前值，则直接判定认证失败：

```
if (*cert_nv_ctr < plat_nv_ctr) {  
    return 1;  
}
```

这里的关键点在于，ATF 并不把“签名合法”视作唯一的信任条件。一个旧版本的证书即使签名完全正确，它依然可能是不安全的，因为旧版本固件中可能存在后来已知并已修复的漏洞。如果攻击者可以把系统回滚到旧版本的合法镜像，那么他就可能重新利用这些漏洞。NV Counter 的作用，就是在系统中记录一个单调增加的安全版本边界，让旧版本即便签名合法也无法再次被接受。

因此，NV Counter 机制补足了数字签名的局限性。签名回答的是“这个对象是不是由合法签发者生成的”，而 NV Counter 回答的是“这个对象是不是属于当前允许接受的版本范围”。两者结合后，可信启动才真正具备了真实性和时序安全性。对于课程作业来说，这一部分也是可以重点强调的，因为它体现出 ATF 的安全设计不是停留在最表面的验签层面，而是进一步考虑了现实攻击场景中的回滚问题。

1.3.7 mbedTLS 中的底层验签与哈希计算

在 `drivers/auth/mbedtls/mbedtls_crypto.c` 中，ATF 使用 `mbedtls` 来完成底层密码学处理。以 `verify_signature()` 为例，其流程大致包括以下几个阶段：首先解析 ASN.1/DER 编码中的签名算法信息，确定摘要算法和公钥算法；然后解析 DER 编码的公钥；接着提取签名比特串；之后对待验证的数据重新计算哈希；最后调用 `mbedtls` 的公钥验签接口完成实际验证。核心代码如下：

```
md_info = mbedtls_md_info_from_type(md_alg);
rc = mbedtls_md(md_info, p, data_len, hash);
rc = mbedtls_pk_verify_ext(pk_alg, sig_opts, &pk, md_alg,
    hash,
    mbedtls_md_get_size(md_info),
    signature.p, signature.len);
```

这段代码清楚地说明，ATF 本身不负责去实现 RSA、ECDSA 或者各种摘要算法的数学过程，而是把这些成熟且复杂的工作交给 `mbedtls` 完成。这种做法有两个好处。第一，可以复用成熟的密码学实现，减少 ATF 自己实现底层算法所带来的风险；第二，可以让认证框架专注于“认证逻辑组织”，而不是陷入密码学细节实现。也就是说，ATF 在架构上刻意把“安全策略”和“密码实现”分层处理了。

同一文件中的 `verify_hash()` 用于哈希比对，其主要步骤是解析输入中的哈希算法和期望哈希值，然后对当前数据重新计算哈希，最后使用 `memcmp` 比较两者是否一致。这个过程虽然看起来比签名验证简单，但在镜像认证中同样重要，因为对原始镜像来说，证书中的哈希值实际上就是“由已认证父对象提供的可信完整性基准”。一旦重新计算的哈希与证书中保存的值不同，就意味着镜像在存储、传输或加载过程中发生了修改，从而应立即拒绝。

从整个可信启动链条来看，`mbedtls` 并不决定“该验证什么”，而只负责“如何验证”。真正决定当前对象的公钥来源、摘要对象、签名算法参数和是否需要哈希或 NV Counter 检查的，是前面的认证框架和 CoT 描述。而 `mbedtls` 是最终的执行者，它接收这些已经组织好的参数，然后输出密码学意义上的真伪判断。这种角色划分在工程上是很合理的，也体现出 ATF 架构设计的清晰性。

1.3.8 BL1 认证 BL2 的完整运行过程总结

综合以上源码分析，可以把 BL1 认证 BL2 的完整运行过程描述为：系统上电后，BL1 先完成平台和体系结构相关初始化，然后初始化密码模块与认证模块。接着，BL1 在 `bl1_load_bl2()` 中调用 `load_auth_image(BL2_IMAGE_ID, info)`，进入认证加载路径。认证加载函数首先查询 BL2 的父节点，根据 CoT 配置得知其父节点为 `trusted_boot_fw_cert`，于是递归转入根证书的认证流程。

在根证书认证阶段，ATF 先解析证书中的被签名数据、签名值和签名算法，然后从平台中获取根信任公钥或其哈希，并验证证书中的公钥是否与平台根信任一致。在此基础上，ATF 调用底层密码模块完成证书签名验证。签名验证通过后，系统继续检查证书中的 NV Counter 是否不小于平台当前安全计数器，以防止加载旧版本证书。只有当签名和计数器两项都通过之后，ATF 才会把根证书中的哈希值、公钥等可信参数提取到缓冲区，作为后续对象认证的依据。

之后，系统回到 BL2 的认证流程，把 BL2 镜像加载到内存中，并调用 `auth_mod_verify_img()` 对其进行认证。根据 CoT 描述，BL2 的认证方式是 `AUTH_METHOD_HASH`，因此 ATF 会取出先前从根证书中提取出的 BL2 哈希值，再对当前加载到内存中的 BL2 内容重新计算哈希并进行比较。如果两者一致，则说明 BL2 的内容与证书中记录的一致，镜像完整且可信，BL1 可以继续把控制权交给 BL2；如果两者不一致，则说明 BL2 已被篡改，ATF 会立即清除镜像内存并终止流程。

通过这一过程可以看出，ATF 不是单点验签，而是在启动阶段建立了一条真正意义上的信任链：平台根信任首先约束根证书，根证书再为 BL2 提供可信哈希，BL2 通过哈希比对获得完整性保证，随后这条信任链又会继续延伸到 BL31、BL32 和 BL33。这个过程既体现了密码学上的认证逻辑，也体现了安全世界启动控制权如何被谨慎地逐步交接。

1.3.9 流程图

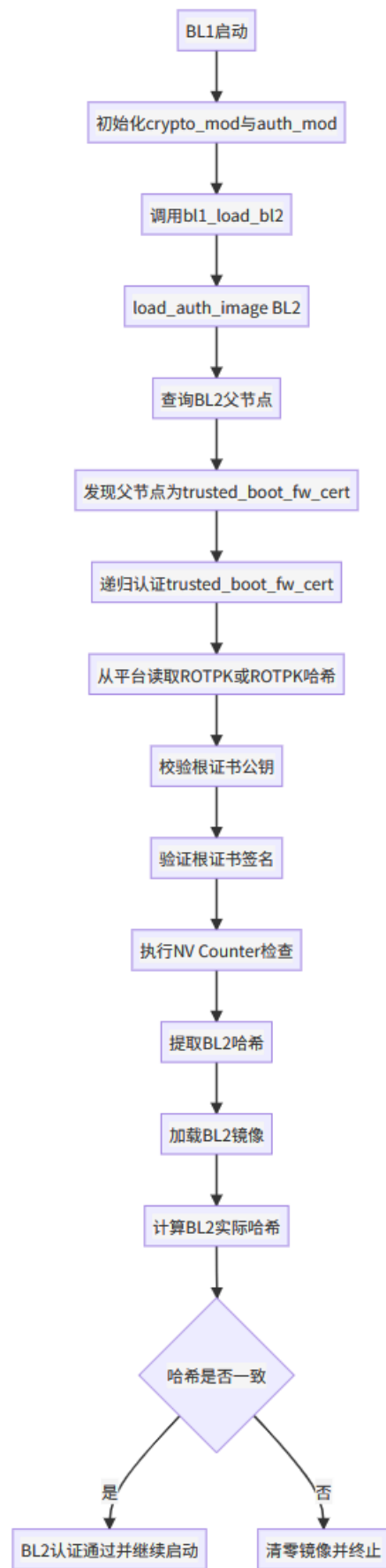


图 1 BL1 认证 BL2 的总体流程

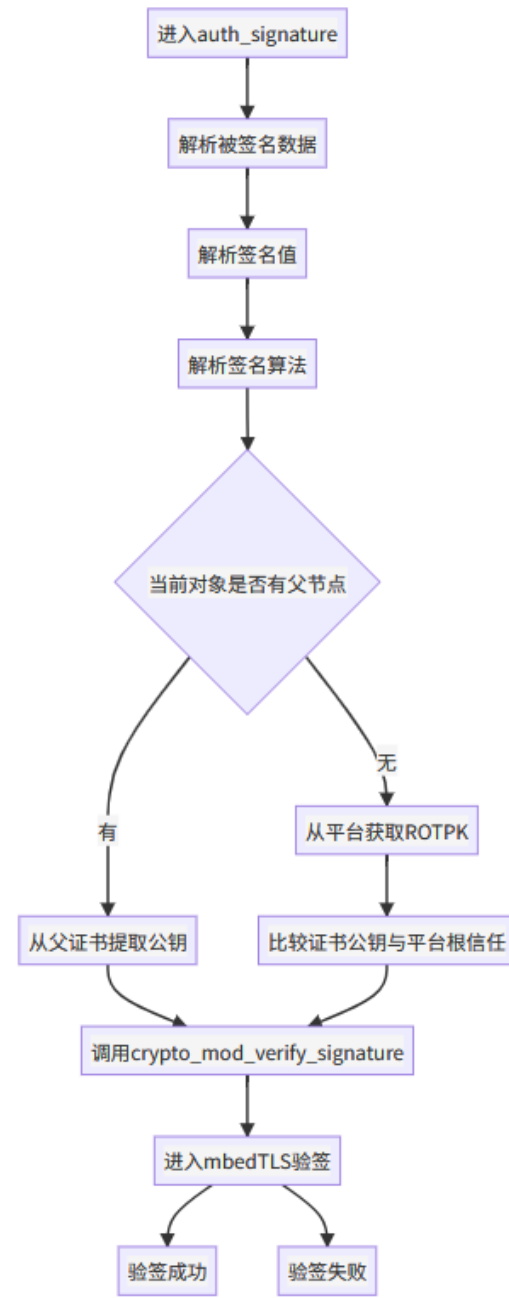


图 2 auth_signature() 的运行流程

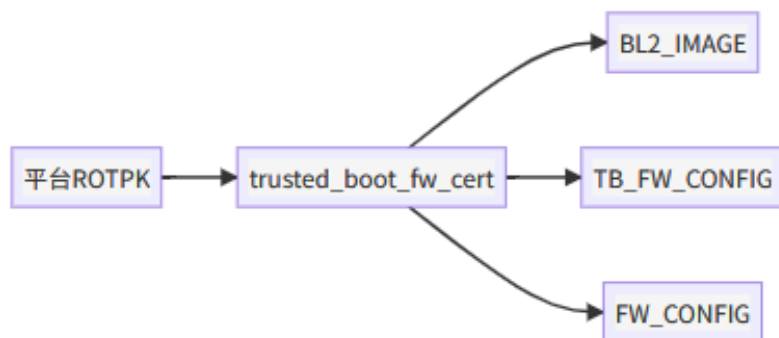


图 3 BL1 阶段信任链关系示意

1.4 分析结论

通过对 ATF 可信启动与签名认证相关源代码的分析，可以看出 ATF 的安全设计远比“简单验签一个镜像”更完整。它从 BL1 启动阶段开始，就把密码能力、认证能力、镜像加载、证书链校验和错误清理整合在一起，形成了一条层次清晰的启动信任链。平台中预置的根信任公钥为整条信任链提供起点，根证书通过签名和 NV Counter 建立可信性，根证书中提取出的哈希和公钥又继续作为下一级对象认证的依据。随后，原始镜像通过哈希比对获得完整性验证，而在认证失败时，ATF 还会立刻清除内存中的镜像数据，以避免不可信对象残留。由此可以看出，ATF 的可信启动机制既考虑了认证过程本身，也考虑了认证失败后的系统安全状态。

通过这次分析，我对 ATF 技术有了更深入的认识。首先，ATF 并不只是一个把不同引导阶段串联起来的固件框架，它本质上是 ARM TrustZone 启动安全体系中的核心基础软件。TrustZone 能够提供安全世界，但安全世界是否真的可信，前提是启动进入安全世界的这条路径本身可信，而这恰恰是 ATF 可信启动机制要解决的问题。其次，ATF 采用的链式信任模型体现了现代安全启动系统的基本思想，即从硬件根信任出发，逐级认证证书和镜像，把“可信”一步一步向下传递，而不是凭空相信后续组件。再次，ATF 把签名验证、哈希验证和 NV Counter 防回滚结合在一起，说明一个完整的安全启动系统不仅要回答“这是谁发布的”“内容有没有被改”，还要回答“是否允许回退到这个版本”。最后，ATF 通过 CoT 描述文件和通用认证框架，把策略定义、平台依赖和密码学实现很好地分离开来，这体现出它不仅是安全设计合理，而且工程结构也相当成熟。

总体来说，ATF 的可信启动机制很好地体现了 TrustZone 和可信固件的核心思想，即在系统最早阶段先在安全世界中建立信任根，再通过受控的认证链把信任安全地传递到后续所有关键固件。我通过阅读 ATF 源代码，更清楚地理解移动终端和嵌入式平台在启动早期是如何建立系统可信基础的，也能够认识到可信启动为什么会成为整个系统安全体系中最根本、最不可替代的一环。

二、思考题

2.1 目标

基于 QEMU 虚拟机，搭建一个 ATF 虚拟运行环境，分别登录 Secure World 及 Normal World 的 linux 系统，并启动命令行 shell。

2.2 相关实验过程

2.2.1 安装前期依赖

1. 安装依赖

```
sudo apt update
sudo apt install -y \
git \
make \
gcc-aarch64-linux-gnu \
binutils-aarch64-linux-gnu \
device-tree-compiler \
qemu-system-arm \
libssl-dev \
bc \
bison \
flex \
cpio \
gzip \
rsync \
python3
```

```
ubuntu@ubuntu-virtual-machine:~/Desktop$ sudo apt update
Hit:1 http://security.ubuntu.com/ubuntu focal-security InRelease
Hit:2 http://us.archive.ubuntu.com/ubuntu focal InRelease
Hit:3 http://us.archive.ubuntu.com/ubuntu focal-updates InRelease
Hit:4 http://us.archive.ubuntu.com/ubuntu focal-backports InRelease
Reading package lists... Done
Building dependency tree
Reading state information... Done
436 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

```

ubuntu@ubuntu-virtual-machine:~/sktop$ sudo apt install -y \
> git \
> make \
> gcc-aarch64-linux-gnu \
> binutils-aarch64-linux-gnu \
> device-tree-compiler \
> qemu-system-arm \
> libssl-dev \
> bc \
> bison \
> flex \
> cpio \
> gzip \
> rsync \
> python3
Reading package lists... Done
Building dependency tree
Reading state information... Done
bc is already the newest version (1.07.1-2build1).
bc set to manually installed.
make is already the newest version (4.2.1-1.2).
make set to manually installed.
python3 is already the newest version (3.8.2-0ubuntu2).
git is already the newest version (1:2.25.1-1ubuntu3.14).
gzip is already the newest version (1:10-0ubuntu4.1).
The following additional packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu cpp-9 gcc-aarch64-linux-gnu cpp-aarch64-linux-gnu gcc-10-cross-base
  gcc-9-cross-base gcc-9-aarch64-linux-gnu gcc-9-base gcc-9-cross-base libverbs-providers ipxe-qemu
  ipxe-qemu-256k-compatible-efi-roms libaio1 libasan5-arm64-cross libatomic1-arm64-cross libbinutils libboost-iostreams1.71.0
  libb6-arm64-cross libb6-dev-arm64-cross libb6carm0 libctf0 libfdt1 libfl1 libfl2 libgcc-9-dev-arm64-cross
  libgcc-s1-arm64-cross libgomp1-arm64-cross liblibverbs1 liblscc17 libltn1-arm64-cross liblsan0-arm64-cross liblpmem1 liblrados2
  liblrbdmac1 libtstgdev2 libtstg0 libtstg-server1 libssl1.1 libstdc++6-arm64-cross libtsan0-arm64-cross
  libubsan1-arm64-cross libubsydr1parser1 libvirglrenderer1 linux-libc-dev-arm64-cross m4 qemu-block-extra qemu-efi-aarch64
  qemu-efi-arm qemu-system-common qemu-system-data qemu-system-gui qemu-utils sharutils
Suggested packages:
  binutils-doc bison-doc libarchive1 gcc-9-locales cpp-doc flex-doc gcc-9-doc autoconf automake libtool gdb-aarch64-linux-gnu
  gcc-doc streamer1.0-plugins-ugly libssl-doc m4-doc samba vde2 debotstrap openssl-server sharutils-doc bsd-mailx | mailx
The following NEW packages will be installed:
  binutils-aarch64-linux-gnu bison bc cpp-9 gcc-9-aarch64-linux-gnu gcc-9-cross-base gcc-10-cross-base
  libb6-arm64-cross libb6-dev-arm64-cross libb6carm0 libctf0 libfdt1 libfl1 libfl2 libgcc-9-dev-arm64-cross
  libgcc-s1-arm64-cross libgomp1-arm64-cross liblibverbs1 liblscc17 libltn1-arm64-cross liblsan0-arm64-cross liblpmem1 liblrados2
  liblrbdmac1 libtstgdev2 libtstg0 libtstg-server1 libssl1.1 libstdc++6-arm64-cross libtsan0-arm64-cross
  libubsan1-arm64-cross libubsydr1parser1 libvirglrenderer1 linux-libc-dev-arm64-cross m4 qemu-block-extra qemu-efi-aarch64
  qemu-efi-arm qemu-system-common qemu-system-data qemu-system-gui qemu-utils sharutils
0 upgraded, 34 newly installed, 0 to remove and 0 not upgraded.
Need to get 108 MB of archives.
After this operation, 410 MB of additional disk space will be used.
Do you want to continue? [Y/n]

```

2.创建工作目录

```
mkdir -p ~/secure-linux-lab
```

```
cd ~/secure-linux-lab
```

```
ubuntu@ubuntu-virtual-machine:~/Desktop$ mkdir -p ~/secure-linux-lab
ubuntu@ubuntu-virtual-machine:~/Desktop$ cd ~/secure-linux-lab
```

3. 下载源码

```
git clone https://github.com/ARM-software/arm-trusted-firmware.git
```

```
git clone https://github.com/torvalds/linux.git
```

```
git clone git://busybox.net/busybox.git
```

```
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab$ git clone git://busybox.net/busybox.git
Cloning into 'busybox'...
remote: Enumerating objects: 660, done.
remote: Counting objects: 100% (660/660), done.
remote: Compressing objects: 100% (660/660), done.
remote: Total 120210 (delta 452), reused 0 (delta 0), pack-reused 119550
Receiving objects: 100% (120210/120210), 30.50 MiB | 2.79 MiB/s, done.
Resolving deltas: 100% (96384/96384), done.
```

4.编译 BusyBox

```
cd ~/secure-linux-lab/busybox
```

```
make defconfig
```

```
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab$ cd ~/secure-linux-lab/busybox
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/busybox$
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/busybox$ make defconfig
HOSTCC scripts/basic/fixdep
HOSTCC scripts/basic/split-include
HOSTCC scripts/basic/docproc
GEN include/applets.h
GEN include/usage.h
GEN init/Kbuild
GEN init/Config.in
GEN procps/Kbuild
GEN procps/Config.in
GEN findutils/Kbuild
GEN findutils/Config.in
GEN libpwdgrp/Kbuild
GEN selinux/Kbuild
GEN selinux/Config.in
GEN networking/Kbuild
GEN networking/Config.in
GEN networking/udhcp/Kbuild
GEN networking/udhcp/Config.in
GEN networking/libiproute/Kbuild
GEN util-linux/Kbuild
GEN util-linux/Config.in
GEN util-linux/volume_id/Kbuild
GEN util-linux/volume_id/Config.in
GEN printutils/Kbuild
GEN printutils/Config.in
GEN syslogd/Kbuild
GEN syslogd/Config.in
GEN applets/Kbuild
GEN libbb/Kbuild
GEN libbb/Config.in
GEN libbb/yescrypt/Kbuild
GEN loginutils/Kbuild
GEN loginutils/Config.in
GEN miscutils/Kbuild
GEN miscutils/Config.in
GEN console-tools/Kbuild
GEN console-tools/Config.in
GEN coreutils/Kbuild
GEN coreutils/Config.in
GEN coreutils/libcoreutils/Kbuild
GEN e2fsprogs/Kbuild
GEN e2fsprogs/Config.in
GEN shell/Kbuild
GEN shell/Config.in
GEN klibc-utils/Kbuild
GEN klibc-utils/Config.in
GEN modutils/Kbuild
GEN modutils/Config.in
```

5. (1) 开启静态编译

```
sed -i 's/# CONFIG_STATIC is not set/CONFIG_STATIC=y/' .config
```

(2) 编译

```
make -j$(nproc)
make install
```



```

ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/busybox$ sed -i 's/# CONFIG_STATIC is not set/CONFIG_STATIC=y/' .config
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/busybox$ make -j$(nproc)
scripts/kconfig/conf -s Config.in
#
# using defaults found in .config
#
SPLIT include/autoconf.h -> include/config/*
GEN include/bbconfigopts.h
GEN include/common_bufsiz.h
GEN include/embedded_scripts.h
HOSTCC applets/usage
HOSTCC applets/applet_tables
applets/usage.c:52:3: warning: ignoring return value of 'write', declared with attribute warn_unused_result [-Wunused-result]
52 | write(STDOUT_FILENO, usage_array[i].usage, strlen(usage_array[i].usage) + 1);
    | ~~~~~^~~~~~
GEN include/usage_compressed.h
GEN include/applet_tables.h include/NUM_APPLETS.h
GEN include/applet_tables.h include/NUM_APPLETS.h
HOSTCC applets/usage_pod
CC applets/applets.o
LD applets/built-in.o
DOC busybox.pod
DOC BusyBox.txt
DOC busybox.1
LD archival/built-in.o
CC archival/bbunzip.o
LD archival/libarchive/built-in.o
CC archival/libarchive/common.o
CC archival/libarchive/data_align.o
CC archival/bzip2.o
CC archival/libarchive/data_extract_all.o
archival/libarchive/data_extract_all.c: In function 'data_extract_all':
archival/libarchive/data_extract_all.c:242:4: warning: ignoring return value of 'chown', declared with attribute warn_unused_result [-Wunused-result]
242 |     chown(dst_name, uid, gid);
    |     ~~~~~^~~~~~
CC archival/libarchive/data_extract_to_command.o
CC archival/libarchive/data_extract_to_stdout.o
CC archival/libarchive/data_skip.o
CC archival/libarchive/decompress_bunzip2.o
CC archival/chksum_and_xwrite_tar_header.o
CC archival/cpio.o

```

```

ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/busybox$ make install
./_install//bin/arch -> busybox
./_install//bin/ash -> busybox
./_install//bin/base32 -> busybox
./_install//bin/base64 -> busybox
./_install//bin/cat -> busybox
./_install//bin/chattr -> busybox
./_install//bin/chgrp -> busybox
./_install//bin/chmod -> busybox
./_install//bin/chown -> busybox
./_install//bin/conspv -> busybox

```

```

-----
You will probably need to make your busybox binary
setuid root to ensure all configured applets will
work properly.
-----

```

6.制作 rootfs

```

cd _install
mkdir -p proc sys dev etc tmp

```

```

ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/busybox$ cd _install
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/busybox/_install$ mkdir -p proc sys dev etc tmp

```

7.创建/init

```

cat > init <<'EOF'
#!/bin/sh
mount -t proc none /proc
mount -t sysfs none /sys
mount -t devtmpfs none /dev

echo ""

```

```
echo "=== SECURE LINUX BOOTED ==="
echo ""

exec /bin/sh

EOF

chmod +x init
```

```
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/busybox/_install$ cat > init <<'EOF'
> #!/bin/sh
> mount -t proc none /proc
> mount -t sysfs none /sys
> mount -t devtmpfs none /dev
>
> echo ""
> echo "=== SECURE LINUX BOOTED ==="
> echo ""
>
> exec /bin/sh
> EOF
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/busybox/_install$ chmod +x init
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/busybox/_install$
```

8.编译 Secure Linux 内核

```
cd ~/secure-linux-lab/linux
make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- defconfig
```

```
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/busybox/_install$ cd ~/secure-linux-lab/linux
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/linux$ make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- defconfig
HOSTCC scripts/basic/fixdep
HOSTCC scripts/kconfig/conf.o
HOSTCC scripts/kconfig/confdata.o
HOSTCC scripts/kconfig/expr.o
LEX scripts/kconfig/lexer.lex.c
YACC scripts/kconfig/parser.tab.[ch]
HOSTCC scripts/kconfig/lexer.lex.o
HOSTCC scripts/kconfig/menu.o
HOSTCC scripts/kconfig/parser.tab.o
HOSTCC scripts/kconfig/preprocess.o
HOSTCC scripts/kconfig/symbol.o
HOSTCC scripts/kconfig/util.o
HOSTLD scripts/kconfig/conf
*** Default configuration is based on 'defconfig'
#
# configuration written to .config
#
```

9.修改相关配置

```
scripts/config --set-val CONFIG_BLK_DEV_INITRD y
scripts/config --set-val CONFIG_DEVTMPFS y
scripts/config --set-val CONFIG_DEVTMPFS_MOUNT y
scripts/config --set-val CONFIG_SERIAL_AMBA_PL011 y
scripts/config --set-val CONFIG_SERIAL_AMBA_PL011_CONSOLE y
scripts/config --set-val CONFIG_TMPFS y
scripts/config --set-val CONFIG_PRINTK y
scripts/config --set-val CONFIG_BINFMT_ELF y
scripts/config --set-val CONFIG_PROC_FS y
scripts/config --set-val CONFIG_SYSFS y
```

10.写死启动参数

```
scripts/config --set-str CONFIG_CMDLINE "console=ttyAMA0 rdinit=/init
earlycon"
scripts/config --enable CONFIG_CMDLINE_FORCE
```

11.把 rootfs 编进 kernel

```
scripts/config --set-str CONFIG_INITRAMFS_SOURCE "/home/$USER/secure-
linux-lab/busybox/_install"
```

```
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/linux$ scripts/config --set-val CONFIG_BLK_DEV_INITRD y
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/linux$ scripts/config --set-val CONFIG_DEVTMPFS y
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/linux$ scripts/config --set-val CONFIG_DEVTMPFS_MOUNT y
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/linux$ scripts/config --set-val CONFIG_SERIAL_AMBA_PL011 y
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/linux$ scripts/config --set-val CONFIG_SERIAL_AMBA_PL011_CONSOLE y
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/linux$ scripts/config --set-val CONFIG_TMPFS y
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/linux$ scripts/config --set-val CONFIG_PRINTK y
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/linux$ scripts/config --set-val CONFIG_BINfmt_ELF y
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/linux$ scripts/config --set-val CONFIG_PROC_FS y
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/linux$ scripts/config --set-val CONFIG_SYSFS y
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/linux$ scripts/config --set-str CONFIG_CMDLINE "console=ttyAMA0 rdinit=/init early
con"
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/linux$ scripts/config --enable CONFIG_CMDLINE_FORCE
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/linux$ scripts/config --set-str CONFIG_INITRAMFS_SOURCE "/home/$USER/secure-linux-
lab/busybox/_install"
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/linux$
```

12.同步配置

```
make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- olddefconfig
```

```
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/linux$ make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- olddefconfig
#
# configuration written to .config
#
```

13.编译

```
make -j$(nproc) ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- Image
```

```
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/linux$ make -j$(nproc) ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- Image
SYNC    include/config/auto.conf.cmd
WRAP    arch/arm64/include/generated/uapi/asm/kvm_para.h
WRAP    arch/arm64/include/generated/uapi/asm/errno.h
WRAP    arch/arm64/include/generated/uapi/asm/ioctl.h
WRAP    arch/arm64/include/generated/uapi/asm/ioctls.h
WRAP    arch/arm64/include/generated/uapi/asm/ipcbuf.h
WRAP    arch/arm64/include/generated/uapi/asm/msgbuf.h
WRAP    arch/arm64/include/generated/uapi/asm/poll.h
WRAP    arch/arm64/include/generated/uapi/asm/resource.h
WRAP    arch/arm64/include/generated/uapi/asm/sembuf.h
WRAP    arch/arm64/include/generated/uapi/asm/shmbuf.h
WRAP    arch/arm64/include/generated/uapi/asm/signalfd.h
```

14.OBJCOPY arch/arm64/boot/Image 可以看到编译成功

```
rm      .tmp_vmlinux2.syms
KSYMS   .tmp_vmlinux2.kallsyms.S
AS       .tmp_vmlinux2.kallsyms.o
LD       vmlinux.unstripped
NM       System.map
SORTTAB vmlinux.unstripped
OBJCOPY vmlinux
GEN      modules.builtin.modinfo
GEN      modules.builtin
OBJCOPY arch/arm64/boot/Image
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/linux$
```

15.把内核复制出来

```
cd ~/secure-linux-lab
cp linux/arch/arm64/boot/Image Image_secure
```

```
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/linux$
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/linux$ cd ~/secure-linux-lab
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab$ cp linux/arch/arm64/boot/Image Image_secure
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab$
```

16.启动 QEMU

```
qemu-system-aarch64 \  
-machine virt \  
-cpu cortex-a57 \  
-m 1024 \  
-nographic \  
-kernel Image_secure
```

```
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/linux$ cd ~/secure-linux-lab  
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab$ cp linux/arch/arm64/boot/Image Image_secure  
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab$ qemu-system-aarch64 \  
> -machine virt \  
> -cpu cortex-a57 \  
> -m 1024 \  
> -nographic \  
> -kernel Image_secure  
[ 0.000000] Booting Linux on physical CPU 0x0000000000 [0x411fd070]  
[ 0.000000] Linux version 7.0.0-rc6 (ubuntu@ubuntu-virtual-machine) (aarch64-linux-gnu-gcc (Ubuntu 9.4.0-1ubuntu1-20.04.2) 9.4.0  
GNU ld (GNU Binutils for Ubuntu) 2.34) #3 SMP PREEMPT Mon Mar 30 02:19:48 PDT 2026  
[ 0.000000] KASLR disabled due to lack of seed  
[ 0.000000] Machine model: linux,dummy-virt  
[ 0.000000] efi: UEFI not found.  
[ 0.000000] earlycon: pl11 at MMIO 0x0000000090000000 (options '')  
[ 0.000000] printk: legacy bootconsole [pl11] enabled  
[ 0.000000] OF: reserved mem: Reserved memory: No reserved-memory node in the DT  
[ 0.000000] NUMA: Faking a node at [mem 0x0000000040000000-0x000000007fffffff]
```

```
[ 5.115235] (0): "0.flash"  
[ 5.118548] (1): "0.flash"  
[ 5.119785] into device "0.flash"  
[ 5.646745] tun: Universal TUN/TAP device driver, 1.6  
[ 5.771706] thunder_xcv, ver 1.0  
[ 5.774566] thunder_bgx, ver 1.0  
[ 5.777387] nicpf, ver 1.0  
[ 5.801108] e1000: Intel(R) PRO/1000 Network Driver  
[ 5.802330] e1000: Copyright (c) 1999-2006 Intel Corporation.  
[ 5.804372] e1000e: Intel(R) PRO/1000 Network Driver  
[ 5.806719] e1000e: Copyright(c) 1999 - 2015 Intel Corporation.  
[ 5.809704] igb: Intel(R) Gigabit Ethernet Network Driver  
[ 5.811126] igb: Copyright (c) 2007-2014 Intel Corporation.  
[ 5.813597] igbvf: Intel(R) Gigabit Virtual Function Network Driver  
[ 5.815224] igbvf: Copyright (c) 2009 - 2012 Intel Corporation.  
[ 5.821319] sky2: driver version 1.30  
[ 5.845266] VFIO - User Level meta-driver version: 0.3  
[ 5.899553] usbcore: registered new interface driver usb-storage  
[ 5.970928] rtc-pl031 9010000.pl031: registered as rtc0  
[ 5.976175] rtc-pl031 9010000.pl031: setting system clock to 2026-03-30T12:19:29 UTC (1774873169)  
[ 5.995888] i2c_dev: i2c /dev entries driver  
[ 6.128458] sdhci: Secure Digital Host Controller Interface driver  
[ 6.131920] sdhci: Copyright(c) Pierre Ossman  
[ 6.145282] Synopsys Designware Multimedia Card Interface Driver  
[ 6.173373] sdhci-pltfm: SDHCI platform and OF driver helper  
[ 6.254918] usbcore: registered new interface driver usbhid  
[ 6.256530] usbhid: USB HID core driver  
[ 6.312187] hw perfevents: enabled with armv8_pmu3 PMU driver, 5 (0,8000000f) counters available  
[ 6.387364] NET: Registered PF_PACKET protocol family  
[ 6.394852] 9pnet: Installing 9P2000 support  
[ 6.397886] Key type dns_resolver registered  
[ 6.603510] registered taskstats version 1  
[ 6.614744] Loading compiled-in X.509 certificates  
[ 6.785808] Demotion targets for Node 0: null  
[ 6.847160] input: gpio-keys as /devices/platform/gpio-keys/input/input0  
[ 6.894901] clk: Disabling unused clocks  
[ 6.897809] PM: genpd: Disabling unused power domains  
[ 6.900457] Warning: unable to open an initial console.  
[ 7.248564] Freeing unused kernel memory: 13376K  
[ 7.255938] Run /init as init process  
/bin/sh: can't access tty; job control turned off  
~ #
```

17.输入 uname -a、ls 相关指令查看信息。

```
~ # uname -a  
Linux (none) 7.0.0-rc6 #6 SMP PREEMPT Mon Mar 30 05:16:33 PDT 2026 aarch64 GNU/Linux  
~ # ls  
bin      dev      init     linuxrc  sbin     usr  
~ #
```

2.2.2 查看搭建的 ATF 虚拟运行环境

1. 输入以下命令查看

```
cd ~/secure-linux-lab/arm-trusted-firmware
```

```
qemu-system-aarch64 \  
-machine virt,secure=on \  
-cpu cortex-a57 \  
-m 1024 \  
-nographic \  
-bios build/qemu/debug/qemu_fw.bios \  
| tee ~/secure-linux-lab/atf_log.txt
```

```
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab$ cd ~/secure-linux-lab/arm-trusted-firmware  
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/arm-trusted-firmware$  
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/arm-trusted-firmware$ qemu-system-aarch64 \  
> -machine virt,secure=on \  
> -cpu cortex-a57 \  
> -m 1024 \  
> -nographic \  
> -bios build/qemu/debug/qemu_fw.bios \  
> | tee ~/secure-linux-lab/atf_log.txt  
NOTICE: Booting Trusted Firmware  
NOTICE: BL1: v2.14.0(debug):  
NOTICE: BL1: Built : 05:35:26, Mar 30 2026  
INFO: BL1: RAM 0xe0ee000 - 0xef6000  
INFO: BL1: cortex_a57: CPU workaround for erratum 3 was applied  
INFO: BL1: cortex_a57: CPU workaround for erratum 99999 was applied  
WARNING: BL1: cortex_a57: CPU workaround for erratum 826974 was missing!  
WARNING: BL1: cortex_a57: CPU workaround for erratum 826977 was missing!  
WARNING: BL1: cortex_a57: CPU workaround for erratum 828024 was missing!  
WARNING: BL1: cortex_a57: CPU workaround for erratum 829520 was missing!  
WARNING: BL1: cortex_a57: CPU workaround for erratum 833471 was missing!  
WARNING: BL1: cortex_a57: CPU workaround for erratum 859972 was missing!  
WARNING: BL1: cortex_a57: CPU workaround for erratum 1319537 was missing!  
INFO: BL1: cortex_a57: CPU workaround for CVE 2017_5715 was applied  
INFO: BL1: cortex_a57: CPU workaround for CVE 2018_3639 was applied  
INFO: BL1: cortex_a57: CPU workaround for CVE 2022_23960 was applied  
INFO: BL1: Loading BL2  
INFO: Loading image id=1 at address 0xe05b000  
INFO: Image id=1 loaded: 0xe05b000 - 0xe063204  
NOTICE: BL1: Booting BL2  
INFO: Entry point address = 0xe05b000  
INFO: SPSR = 0x3c5  
NOTICE: BL2: v2.14.0(debug):  
NOTICE: BL2: Built : 05:35:46, Mar 30 2026  
INFO: BL2: Doing platform setup  
INFO: BL2: Loading image id 3  
INFO: Loading image id=3 at address 0xe090000  
INFO: Image id=3 loaded: 0xe090000 - 0xe09e89b  
INFO: BL2: Loading image id 5  
INFO: Loading image id=5 at address 0x60000000
```

```

NOTICE: BL1: Booting BL31
INFO:   Entry point address = 0xe090000
INFO:   SPSR = 0x3cd
NOTICE: BL31: v2.14.0(debug):
NOTICE: BL31: Built : 05:35:59, Mar 30 2026
INFO:   ARM GICv2 driver initialized
INFO:   BL31: Initializing runtime services
INFO:   BL31: cortex_a57: CPU workaround for erratum 3 was applied
INFO:   BL31: cortex_a57: CPU workaround for erratum 99999 was applied
WARNING: BL31: cortex_a57: CPU workaround for erratum 826974 was missing!
WARNING: BL31: cortex_a57: CPU workaround for erratum 826977 was missing!
WARNING: BL31: cortex_a57: CPU workaround for erratum 828024 was missing!
WARNING: BL31: cortex_a57: CPU workaround for erratum 829520 was missing!
WARNING: BL31: cortex_a57: CPU workaround for erratum 833471 was missing!
WARNING: BL31: cortex_a57: CPU workaround for erratum 859972 was missing!
WARNING: BL31: cortex_a57: CPU workaround for erratum 1319537 was missing!
INFO:   BL31: cortex_a57: CPU workaround for CVE 2017_5715 was applied
INFO:   BL31: cortex_a57: CPU workaround for CVE 2018_3639 was applied
INFO:   BL31: cortex_a57: CPU workaround for CVE 2022_23960 was applied

```

2.在实验中,通过 QEMU 启动 TF-A 后,终端输出显示系统依次执行 BL1、BL2、BL31 启动阶段。其中 BL31 为 EL3 Runtime Firmware,负责 Secure Monitor 功能。当 BL31 成功初始化并输出相关日志时,说明系统已经成功进入 ATF 虚拟运行环境。BL31 的启动表明系统已经进入 EL3 安全级别,具备管理 Secure World 与 Normal World 切换的能力。

2.2.3 登录 Secure World 的 linux 系统并启动命令行 shell

1.首先准备 TF-A, 编译 TF-A

```

cd ~/secure-linux-lab/arm-trusted-firmware

make distclean

make PLAT=qemu \
    CROSS_COMPILE=aarch64-linux-gnu- \
    SPD=opteed \
    BL32=~/optee-manual/optee_os/out/arm-plat-vexpress/core/tee-
header_v2.bin \
    BL32_EXTRA1=~/optee-manual/optee_os/out/arm-plat-vexpress/core/tee-
pager_v2.bin \
    BL32_EXTRA2=~/optee-manual/optee_os/out/arm-plat-vexpress/core/tee-
pageable_v2.bin \
    BL32_RAM_LOCATION=tdram \
    BL33=./Image_secure \
    DEBUG=1 \
    all fip

```



```

ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/arm-trusted-firmware$ cd ~/secure-linux-lab/arm-trusted-firmware
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/arm-trusted-firmware$
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/arm-trusted-firmware$ make distclean
Arm Architecture Features specified: crc
REALCLEAN
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/arm-trusted-firmware$
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/arm-trusted-firmware$ make PLAT=qemu \
> CROSS_COMPILE=aarch64-linux-gnu- \
> SPD=opteed \
> BL32=~/optee-manual/optee_os/out/arm-plat-vexpress/core/tee-header_v2.bin \
> BL32_EXTRA1=~/optee-manual/optee_os/out/arm-plat-vexpress/core/tee-pager_v2.bin \
> BL32_EXTRA2=~/optee-manual/optee_os/out/arm-plat-vexpress/core/tee-pageable_v2.bin \
> BL32_RAM_LOCATION=tdram \
> BL33=../Image_secure \
> ARM_LINUX_KERNEL_AS_BL33=1 \
> RESET_TO_BL31=1 \
> DEBUG=1 \
> all fip
Including services/spd/opteed/opteed.mk
MD      /home/ubuntu/secure-linux-lab/arm-trusted-firmware/build/qemu/debug/libfdt
CC      lib/libfdt/fdt.c
CC      lib/libfdt/fdt_ro.c
CC      lib/libfdt/fdt_wip.c
CC      lib/libfdt/fdt_sw.c
CC      lib/libfdt/fdt_rw.c
CC      lib/libfdt/fdt_strerror.c
CC      lib/libfdt/fdt_empty_tree.c
CC      lib/libfdt/fdt_addresses.c
CC      lib/libfdt/fdt_overlay.c
CC      lib/libfdt/fdt_check.c
MD      /home/ubuntu/secure-linux-lab/arm-trusted-firmware/build/qemu/debug/lib
AR      /home/ubuntu/secure-linux-lab/arm-trusted-firmware/build/qemu/debug/lib/libfdt.a
MD      /home/ubuntu/secure-linux-lab/arm-trusted-firmware/build/qemu/debug/bl1
CC      bl1/aarch64/bl1_arch_setup.c
CC      bl1/aarch64/bl1_context_mgmt.c
CC      bl1/bl1_main.c
CC      common/backtrace/backtrace.c
CC      common/bl_common.c
CC      common/tf_log.c
CC      drivers/console/multi_console.c
CC      drivers/io/io_fip.c
CC      drivers/io/io_memmap.c
CC      drivers/io/io_semihosting.c
CC      drivers/io/io_storage.c
CC      lib/compiler-rt/builtins/popcountdi2.c
CC      lib/compiler-rt/builtins/popcountsi2.c
CC      lib/cpus/errata_common.c
CC      lib/cpus/errata_report.c
CC      lib/el3_runtime/aarch64/context_mgmt.c
CC      lib/extensions/pmu3/aarch64/pmu3.c
CC      lib/locks/exclusive/aarch64/spinlock.c

```

2.启动系统

```

qemu-system-aarch64 \
  -machine virt,secure=on \
  -cpu cortex-a57 \
  -m 1024 \
  -nographic \
  -bios build/qemu/debug/qemu_fw.bios

```

```

ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/arm-trusted-firmware$ qemu-system-aarch64 \
> -machine virt,secure=on \
> -cpu cortex-a57 \
> -m 1024 \
> -nographic \
> -bios build/qemu/debug/qemu_fw.bios

```

```
Terminal
===== Secure World =====
NOTICE: Booting Trusted Firmware
NOTICE: BL1: v2.14.0(debug):
NOTICE: BL1: Built : 07:39:48, Mar 30 2026
INFO: BL1: RAM 0xe0ee000 - 0xe0f6000
INFO: BL1: cortex_a57: CPU workaround for erratum 3 was applied
INFO: BL1: cortex_a57: CPU workaround for erratum 99999 was applied
WARNING: BL1: cortex_a57: CPU workaround for erratum 826974 was missing!
WARNING: BL1: cortex_a57: CPU workaround for erratum 826977 was missing!
WARNING: BL1: cortex_a57: CPU workaround for erratum 828024 was missing!
WARNING: BL1: cortex_a57: CPU workaround for erratum 829520 was missing!
WARNING: BL1: cortex_a57: CPU workaround for erratum 833471 was missing!
WARNING: BL1: cortex_a57: CPU workaround for erratum 859972 was missing!
WARNING: BL1: cortex_a57: CPU workaround for erratum 1319537 was missing!
INFO: BL1: cortex_a57: CPU workaround for CVE 2017_5715 was applied
INFO: BL1: cortex_a57: CPU workaround for CVE 2018_3639 was applied
INFO: BL1: cortex_a57: CPU workaround for CVE 2022_23960 was applied
INFO: BL1: Loading BL2
INFO: Loading image id=1 at address 0xe05b000
INFO: Image id=1 loaded: 0xe05b000 - 0xe06442c
NOTICE: BL1: Booting BL2
INFO: Entry point address = 0xe05b000
INFO: SPSR = 0x3c5
NOTICE: BL2: v2.14.0(debug):
NOTICE: BL2: Built : 07:40:10, Mar 30 2026
INFO: BL2: Doing platform setup
INFO: BL2: Loading image id 3
INFO: Loading image id=3 at address 0xe090000
INFO: Image id=3 loaded: 0xe090000 - 0xe0a089b
INFO: BL2: Loading image id 4
INFO: Loading image id=4 at address 0xe100000
INFO: Image id=4 loaded: 0xe100000 - 0xe10001c
INFO: OPTEE ep=0xe100000
INFO: OPTEE header info:
INFO: magic=0x4554504f
INFO: version=0x2
INFO: arch=0x1
```

我们可以看到，成功搭建了 ARM TrustZone 虚拟运行环境。系统启动流程如下：Trusted Firmware-A 成功启动，日志中出现 BL1、BL2、BL31，表明 EL3 固件层正常运行；OP-TEE 作为 BL32 成功加载并运行，日志中出现 optee: revision 等信息，说明 Secure World 已建立；Linux 作为 BL33 成功启动，并进入命令行 shell，能够执行 uname、ls 等命令，说明 Normal World 正常运行。

```
[ 7.638662] 4000000.flash: Found 2 x16 devices at 0x0 in 32-bit bank. Manufacturer ID 0x000000 Chip ID 0x000000
[ 7.673201] Intel/Sharp Extended Query Table at 0x0031
[ 7.718072] Using buffer write method
[ 8.176379] tun: Universal TUN/TAP device driver, 1.6
[ 8.328694] thunder_xcv, ver 1.0
[ 8.336210] thunder_bgx, ver 1.0
[ 8.346030] nicpf, ver 1.0
[ 8.405531] e1000: Intel(R) PRO/1000 Network Driver
[ 8.420459] e1000: Copyright (c) 1999-2006 Intel Corporation.
[ 8.424861] e1000e: Intel(R) PRO/1000 Network Driver
[ 8.432302] e1000e: Copyright(c) 1999 - 2015 Intel Corporation.
[ 8.440532] igb: Intel(R) Gigabit Ethernet Network Driver
[ 8.457043] igb: Copyright (c) 2007-2014 Intel Corporation.
[ 8.474058] igbvf: Intel(R) Gigabit Virtual Function Network Driver
[ 8.483614] igbvf: Copyright (c) 2009 - 2012 Intel Corporation.
[ 8.518739] sky2: driver version 1.30
[ 8.572533] VFIO - User Level meta-driver version: 0.3
[ 8.697451] usbcore: registered new interface driver usb-storage
[ 8.793954] rtc-pl031 9010000.pl031: registered as rtc0
[ 8.814281] rtc-pl031 9010000.pl031: setting system clock to 2026-03-30T14:41:13 UTC (1774881673)
[ 8.876982] i2c_dev: i2c /dev entries driver
[ 9.019812] sdhci: Secure Digital Host Controller Interface driver
[ 9.027610] sdhci: Copyright(c) Pierre Ossman
[ 9.054482] Synopsys Designware Multimedia Card Interface Driver
[ 9.094180] sdhci-pltfm: SDHCI platform and OF driver helper
[ 9.211762] SHCCC: SOC_ID: ARCH_SOC_ID not implemented, skipping ....
[ 9.261495] usbcore: registered new interface driver usbhid
[ 9.278038] usbhid: USB HID core driver
[ 9.379907] hw perfevents: enabled with armv8_pmu3 PMU driver, 5 (0,8000000f) counters available
[ 9.449923] optee: probing for conduit method.
[ 9.476091] optee: revision 4.9
[ 9.521250] optee: Asynchronous notifications enabled
[ 9.546500] optee: dynamic shared memory is enabled
[ 9.692130] optee: initialized driver
[ 9.786060] NET: Registered PF_PACKET protocol family
[ 9.828362] 9pnet: Installing 9P2000 support
[ 9.840286] Key type dns_resolver registered
[ 10.114295] registered taskstats version 1
[ 10.201084] Loading compiled-in X.509 certificates
[ 10.401948] Demotion targets for Node 0: null
[ 10.402085] clk: Disabling unused clocks
[ 10.402096] PM: genpd: Disabling unused power domains
[ 10.565566] Warning: unable to open an initial console.
[ 10.900004] Freeing unused kernel memory: 13376K
[ 10.908345] Run /init as init process
```


3.进入 shell, 输入 ls、uname -a 验证 Secure World。

```
~ # ls
bin      dev      init     linuxrc  sbin     usr
~ # uname -a
Linux (none) 7.0.0-rc6 #7 SMP PREEMPT Mon Mar 30 06:08:21 PDT 2026 aarch64 GNU/Linux
```

成功进入命令行 shell 后, 执行 ls 和 uname -a 命令对 Secure World 中系统的运行状态进行验证。实验结果表明, 系统能够正确响应用户命令, 并输出文件系统结构及内核版本信息, 说明 Secure World 已成功建立且 shell 运行正常。

2.2.4 登录 Normal World 的 linux 系统并启动命令行 shell

1.复制一个正常世界的内核

```
cd ~/secure-linux-lab
cp Image_secure Image_normal
```

```
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab/arm-trusted-firmware$ cd ~/secure-linux-lab
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab$ cp Image_secure Image_normal
```

2.启动 normal linux

```
qemu-system-aarch64 \
-m 1024 \
-cpu cortex-a57 \
-machine virt \
-nographic \
-kernel Image_normal \
-append "console=ttyAMA0 init=/init"
```

```
ubuntu@ubuntu-virtual-machine:~/secure-linux-lab$ qemu-system-aarch64 \
> -machine virt \
> -cpu cortex-a57 \
> -m 1024 \
> -nographic \
> -kernel Image_normal \
> -append "console=ttyAMA0 init=/init"
```

```
Terminal
===== Normal World =====
NOTICE: Booting Trusted Firmware
NOTICE: BL1: v2.14.0(debug):
NOTICE: BL1: Built : 07:39:48, Mar 30 2026
INFO: BL1: RAM 0xe0ee000 - 0xe0f6000
INFO: BL1: cortex_a57: CPU workaround for erratum 3 was applied
INFO: BL1: cortex_a57: CPU workaround for erratum 99999 was applied
WARNING: BL1: cortex_a57: CPU workaround for erratum 826974 was missing!
WARNING: BL1: cortex_a57: CPU workaround for erratum 826977 was missing!
WARNING: BL1: cortex_a57: CPU workaround for erratum 828024 was missing!
WARNING: BL1: cortex_a57: CPU workaround for erratum 829520 was missing!
WARNING: BL1: cortex_a57: CPU workaround for erratum 833471 was missing!
WARNING: BL1: cortex_a57: CPU workaround for erratum 859972 was missing!
WARNING: BL1: cortex_a57: CPU workaround for erratum 1319537 was missing!
INFO: BL1: cortex_a57: CPU workaround for CVE 2017_5715 was applied
INFO: BL1: cortex_a57: CPU workaround for CVE 2018_3639 was applied
INFO: BL1: cortex_a57: CPU workaround for CVE 2022_23960 was applied
INFO: BL1: Loading BL2
INFO: Loading image id=1 at address 0xe05b000
INFO: Image id=1 loaded: 0xe05b000 - 0xe06442c
NOTICE: BL1: Booting BL2
INFO: Entry point address = 0xe05b000
INFO: SPSR = 0x3c5
NOTICE: BL2: v2.14.0(debug):
NOTICE: BL2: Built : 07:40:10, Mar 30 2026
INFO: BL2: Doing platform setup
INFO: BL2: Loading image id 3
INFO: Loading image id=3 at address 0xe090000
INFO: Image id=3 loaded: 0xe090000 - 0xe0a089b
INFO: BL2: Loading image id 4
INFO: Loading image id=4 at address 0xe100000
INFO: Image id=4 loaded: 0xe100000 - 0xe10001c
INFO: OPTEE ep=0xe100000
INFO: OPTEE header info:
INFO: magic=0x4554504f

Starting syslogd: OK
Starting klogd: OK
Running sysctl: OK
Set permissions on /dev/tee*: OK
Create/set permissions on /var/lib/tee: OK
Starting tee-supPLICant: Using device /dev/teepriv0.
*plugin*: init, version: 0.4.0
OK
Starting network: OK
Starting crond: OK
Starting network (udhcpc): OK

Welcome to Buildroot, type root or test to login
buildroot login: root
#
```

3.同样输入 ls、uname -a、hello、cat /proc/version 验证 Normal World 的 shell

```
~ # ls /
bin      dev      init      linuxrc  sbin     usr
~ # uname -a
Linux (none) 7.0.0-rc6 #7 SMP PREEMPT Mon Mar 30 06:08:21 PDT 2026 aarch64 GNU/Linux
~ # echo hello
hello
~ # mkdir -p /proc
~ # /bin/busybox mount -t proc proc /proc
~ # /bin/busybox cat /proc/version
Linux version 7.0.0-rc6 (ubuntu@ubuntu-virtual-machine) (aarch64-linux-gnu-gcc (Ubuntu 9.4.0-1ubuntu1~20.04.2) 9.4.0, GNU ld (GNU Binutils for Ubuntu) 2.34) #7 SMP PREEMPT Mon Mar 30 06:08:21 PDT 2026
~ # ^C
~ # cat /proc/version
Linux version 7.0.0-rc6 (ubuntu@ubuntu-virtual-machine) (aarch64-linux-gnu-gcc (Ubuntu 9.4.0-1ubuntu1~20.04.2) 9.4.0, GNU ld (GNU Binutils for Ubuntu) 2.34) #7 SMP PREEMPT Mon Mar 30 06:08:21 PDT 2026
~ #
```

以上可以说明，Normal World Linux 已成功运行，并能够进入命令行 shell，且系统具备基本的用户空间功能。

2.3 思考题总结

本实验基于 QEMU 虚拟机平台，成功搭建了 ARM TrustZone 架构下的 ATF（ARM Trusted Firmware）虚拟运行环境，并分别验证了 Secure World 与 Normal World 中 Linux 系统的运行情况。

在实验过程中，首先通过编译与配置 Trusted Firmware-A，实现了系统启动链路中的 BL1、BL2、BL31 阶段，验证了 EL3 安全监控层的正常运行。随后，我成功构建 Secure World 环境，并进入命令行 shell 执行相关命令验证了安全世界的初始化与运行。

在 Normal World 部分，将 Linux 内核作为 BL33 加载，并通过 QEMU 成功启动系统，进入命令行 shell。通过执行 `ls` 和 `uname -a` 等命令，验证了系统能够正常响应用户指令并输出文件系统及内核信息，说明非安全世界操作系统运行正常。

综合实验结果可以得出：基于 QEMU 的 ATF 虚拟运行环境已成功搭建，Secure World 与 Normal World 的 linux 系统均已正确建立，并能够进入命令行 shell，实现了基本的系统交互功能。我进一步加深了对 ARM Trust Zone 架构的理解，为后续安全系统开发与研究奠定了基础。